

Aprendizagem Aplicada à Segurança

Class 2: SPAM Detection, from Naive Bayes to Small Language Models

Mário Antunes

September 25, 2026

Universidade de Aveiro

Table of Contents i

The Problem

The Ladder

Text to Numbers

Rung 1-2: Naive Bayes

Rung 3: TF-IDF

Rung 4: Logistic Regression

Rung 5: A Fair Comparison

Rung 6: fastText

Rung 7: Small Language Models

The Spammer Strikes Back

Wrap-up

Table of Contents ii

Appendix: Naive Bayes in Depth

TF-IDF

Logistic Regression

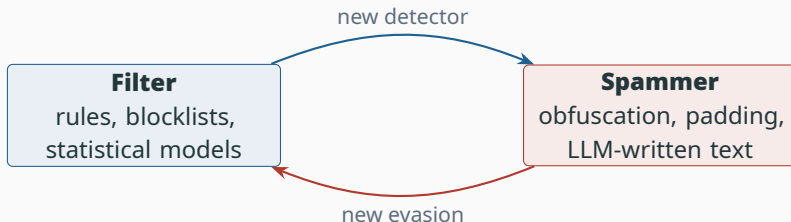
Evaluation

fastText and Embeddings

Attack Mathematics

Spam: an adversary, not a data set

- **Spam** is unsolicited bulk messaging; it carries advertising, fraud, phishing links and malware.
- A filter and a spammer are locked in an **arms race**: every new defence creates a new evasion.



- This class: **build** the detector, rung by rung. Next: **break** it.

From rules to statistics

- **Rules and blocklists** (1990s): keywords, sender lists. Precise, but brittle and easy to dodge (fr33).
- **Bayesian filtering** (2002): learn word statistics from *your own* mail (Graham 2002).
- Then linear models, ensembles, and today **embeddings** from pre-trained language models.

Era	Representation	Typical model
Rules	keywords	if-then
2002-	word counts, TF-IDF	Naive Bayes, logistic regression
2016-	sub-word embeddings	fastText (Joulin et al. 2017)
2019-	sentence embeddings	small language models (Reimers and Gurevych 2019)

The task: binary text classification



- **Positive class = spam** (rare). Two errors, two costs:
 - **false positive:** a real message is blocked (users lose trust);
 - **false negative:** spam reaches the inbox (the filter fails).
- Most of this class is about **how to turn text into $\mathbf{x}$** , and which f_{θ} to use.

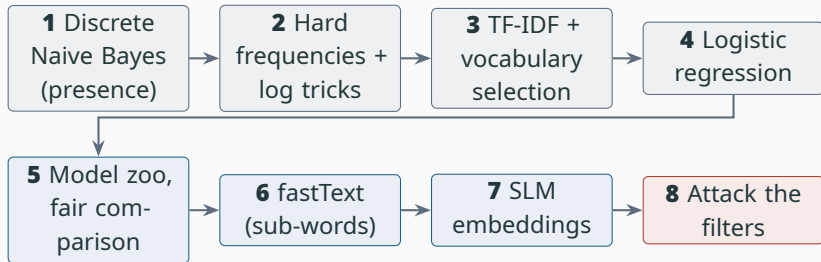
Data and evaluation protocol

SMS Spam Collection (Almeida et al. 2011): 5,574 English SMS, labelled ham or spam.

Step	What we do	Why
De-duplicate	5,574 $\rightarrow$ 5,119 unique	copies in train and test inflate the score
Stratified split	4,095 train, 1,024 test (129 spam)	keeps 12.6% spam in both
Fit on train only	vocabulary, IDF, embeddings	no information from the test set
Metrics	precision, recall, F1, MCC, PR-AUC	accuracy hides the class imbalance

- A filter that says “ham” to everything scores 87% accuracy and catches **no** spam.

From the simplest filter to a language model



- Each rung fixes a **weakness** of the previous one; each has a notebook in notebooks/01-spam/ (00 to 07).
- We keep the **same data, split and metrics** on every rung, so the numbers are comparable.

Tokenisation and normalisation

URGENT! You won a £1000 prize. Call
09061701461 now !!

↓ lower-case, digit runs → num, keep ! and £

urgent ! you won a £ prize call num now ! !

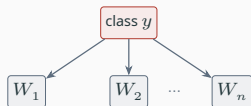
- **Bag of words:** keep *which* words and *how often*, forget the order.
- **Design choices matter:** £, ! and phone-number *shape* are strong spam signals; a naive `isalpha()` filter throws them away.
- Vocabulary from the **training set only** (words in ≥ 2 messages: 3,022).

Bayes' theorem for text

A message is a list of words $W_1, \dots, W_n$; the class is $y \in \{\text{spam}, \text{ham}\}$.

$$P(y \mid W_1..W_n) = \frac{P(y) P(W_1..W_n \mid y)}{P(W_1..W_n)}$$

Naive assumption: words are independent *given the class*, so $P(W_1..W_n \mid y) = \prod_i P(W_i \mid y)$.



Rung 1: discrete Naive Bayes by hand

Presence of a word, Laplace smoothing $k = 1$:

$$P(w \mid y) = \frac{\#\{y\text{-messages containing } w\} + 1}{\#\{y\text{-messages}\} + 2}$$

Toy set: 4 spam, 3 ham; $P(\text{spam}) = 0.571$.

word	$P(w \mid \text{spam})$	$P(w \mid \text{ham})$
password	0.500	0.200
your	0.667	0.400
activity	0.167	0.600
renew (unseen)	0.167	0.200

- send us your password $\rightarrow 0.979$; renew your vows $\rightarrow 0.436$: **wrong** (no meaning).

Rung 2: hard frequencies (counts)

Multinomial model: a message is a **bag of n tokens**; use word *counts* x_w .

$$P(w \mid y) = \frac{N_{w,y} + k}{\sum_{w'} N_{w',y} + k |V|}, \quad \log P(y \mid \mathbf{x}) \propto \log P(y)$$

- The score is a **dot product** of the count vector with a table of log-probabilities.
- On the real corpus (notebooks/01-spam/01):

Model	Precision	Recall	F1	MCC
NB, presence ($k = 1$)	0.953	0.946	0.949	0.942
NB, counts ($k = 1$)	0.961	0.953	0.957	0.951

Logarithms: numerical stability

A 1,007-token message: the product of $\sim 1,000$ probabilities underflows.

Method	$P(\text{spam})$ and $P(\text{ham})$ (unnormalised)	Posterior
Product of probabilities	0.0 and 0.0	$0/0 = \mathbf{NaN}$
Sum of log-probabilities	-6114.6 and -7651.0	≈ 1 (spam)

- $\log ab = \log a + \log b$: **multiplications become sums.**
- Normalise with **log-sum-exp**:
 $\log \sum_y e^{\ell_y} = m + \log \sum_y e^{\ell_y - m}, m = \max_y \ell_y.$
- Never exp a very negative number alone;
`scipy.special.logsumexp` does the shift.

Evaluate like a defender

	predicted ham	predicted spam
actual ham	TN	FP: real message blocked
actual spam	FN: spam in the inbox	TP: spam caught

$$\text{precision} = \frac{TP}{TP + FP} \quad \text{recall} = \frac{TP}{TP + FN} \quad F_1 = \frac{2PR}{P + R}$$

- Users forgive some spam, not a lost message: **push precision up**, read the recall. Naive Bayes at threshold 0.9914: precision **0.992**, recall **0.930**.

Term frequency and inverse document frequency

Counts have two problems: **frequent words dominate** (you, to, the), and **long messages dominate**.

$$\text{tf-idf}(t, d) = (1 + \ln c_{t,d}) \cdot \left(\ln \frac{1+N}{1+\text{df}(t)} + 1 \right), \quad \mathbf{x}_d \leftarrow \frac{\mathbf{x}_d}{\|\mathbf{x}_d\|_2}$$

- **Sub-linear tf** (log): the 10th repetition matters less than the 2nd.
- **idf**: weight by rarity (Spärck Jones 1972). The lowest-idf words: i, to, you, a, the, !, u.
- **L2 normalisation**: every message has length 1, so length no longer matters.

Does TF-IDF help Naive Bayes?

Model	Precision	Recall	F1	PR-AUC
Multinomial NB, counts	0.953	0.953	0.953	0.974
Multinomial NB, TF-IDF	0.968	0.930	0.949	0.983
Complement NB, TF-IDF	0.873	0.961	0.915	0.983

- **Honest result:** F1 is the same within the noise; the *ranking* (PR-AUC) improves.
- χ^2 **vocabulary selection** keeps the informative words: num, p, txt, claim, free, call, mobile, prize, www, stop.
- Those are also the words an attacker must **disguise**.

From generative to discriminative

Naive Bayes models *how each class generates words*. Logistic regression learns **one weight per word** directly:

$$z = \mathbf{w} \cdot \mathbf{x} + b, \quad P(\text{spam} \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- Both give a **linear** score; NB's weights are log-odds $\log \frac{P(w|\text{spam})}{P(w|\text{ham})}$, LR's are optimised.
- Loss (cross-entropy) with L2: $J = -\frac{1}{n} \sum_i [y_i \log \sigma(z_i) + (1 - y_i) \log \sigma(-z_i)] + \frac{\lambda}{2} \|\mathbf{w}\|^2$.
- **Stable** form: `log_sigmoid`; gradient $\frac{1}{n} X^\top (\sigma(X\mathbf{w} + b) - \mathbf{y}) + \lambda \mathbf{w}$, trained with **gradient descent** or **L-BFGS**.

What the weights say

Pushes to spam	weight	Pushes to ham	weight
num (phone number)	20.8	i	−6.0
p (pence)	8.0	my	−4.0
text, txt	7.7, 7.6	me	−3.1
mobile	6.8	gt, lt	−3.0
won, free	5.9, 5.7	can, he	−2.7, −2.5

- gt and lt are HTML escapes (< ; #> ;) in the *ham* half of the corpus: a **spurious correlation**, an artefact of data collection (Arp et al. 2022).
- The weights are also the **attacker's roadmap**.

Which model family?

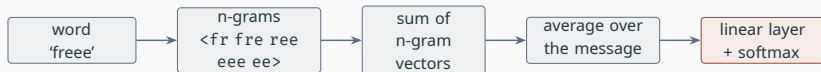
5-fold cross-validation on the training set, TF-IDF *inside* each fold (mean $\pm$ std of F1):

Model	words 1-2 grams	characters 2-5 grams
Multinomial NB	0.947 $\pm$ 0.012	0.946 $\pm$ 0.009
Logistic regression	0.955 $\pm$ 0.015	0.955 $\pm$ 0.009
Linear SVM	0.956 $\pm$ 0.009	0.962 $\pm$ 0.010
Random forest	0.933 $\pm$ 0.016	0.940 $\pm$ 0.016
k-NN (cosine)	0.917 $\pm$ 0.027	0.908 $\pm$ 0.015
MLP (128)	0.914 $\pm$ 0.034	0.932 $\pm$ 0.017

- Differences below the $\pm$ are **noise**. Linear models on sparse TF-IDF are hard to beat.

Sub-words: words are made of pieces

Bag of words has **no similarity** (cash vs money) and **no out-of-vocabulary handling** (fr33).



- Word vectors are the **sum of the vectors of their character n-grams** (Bojanowski et al. 2017); freee shares most n-grams with free.
- The classifier is an **average of embeddings followed by a linear layer**; trains in seconds on a CPU (Joulin et al. 2017).

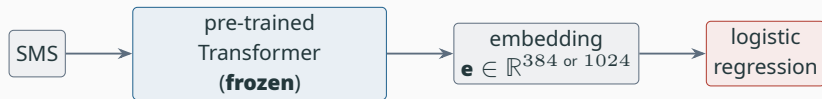
Unseen words still get a useful vector: nearest neighbours of freee: free, freemsg, freefone; of congratulationz: congratulations, congrats.

Model	Precision	Recall	F1	PR-AUC
Skip-gram embeddings + LR	0.991	0.884	0.934	0.981
fastText supervised	0.967	0.915	0.940	0.981
TF-IDF words + LR	0.983	0.907	0.944	0.965
TF-IDF characters + LR	0.983	0.899	0.939	0.987

- On clean text: **on par** with a tuned linear model. The gain is **speed, dense vectors and tolerance to spelling variation**.

Meaning from a pre-trained model

A **small language model** (up to ~1B parameters) already knows what words and sentences mean. Keep it **frozen**, read out one vector, train a tiny classifier (*linear probe*).



Model	Type	Parameter	Dimension
all-MiniLM-L6-v2	encoder, sentence-transformers	22 M	384
Qwen3-Embedding-0.6B (Zhang et al. 2025)	decoder LLM as embedder	600 M	1024

Similar meaning, no shared words

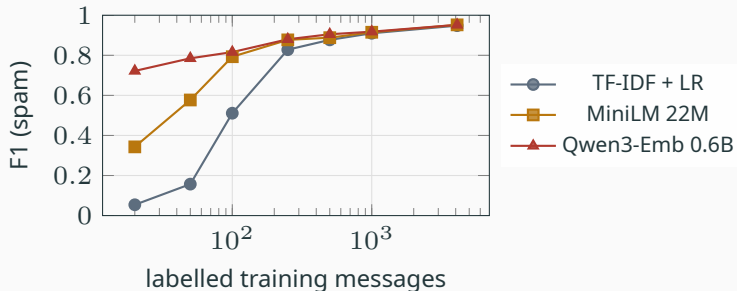
You have won a free holiday, claim your cash prize now vs Congratulations! Collect your complimentary money reward

Representation	cosine similarity
TF-IDF	0.07
MiniLM (22M)	0.53
Qwen3-Embedding (0.6B)	0.97

- TF-IDF sees *two unrelated messages*; embeddings see the **same intent**.
- LLM embeddings are **anisotropic** (all cosines high): standardise each dimension before the probe.

The real benefit: few labels

A new spam campaign has **few labelled examples**. F1 on the same test set, training on n labelled messages:



The price of meaning

Representation + classifier	F1 (full data)	ms per message	Size
TF-IDF words + LR	0.944	0.04	about 1 MB
MiniLM embeddings + LR	0.952	2.8	22 M param
Qwen3-Embedding + LR	0.945	70	600 M param
Qwen3-Embedding + MLP head (Keras)	0.932	70	600 M param

- On abundant data the gap is **within the noise**; the SLM costs **50 to 1,700 times more** per message.
- **Design rule:** start with TF-IDF and a linear model; use an SLM as a second stage, or when labels are scarce.

The ladder in one table

Rung	Model	Precision	Recall	F1
1	NB presence	0.953	0.946	0.949
2	NB counts	0.961	0.953	0.957
3	NB TF-IDF	0.968	0.930	0.949
4	Logistic regression (TF-IDF)	0.975	0.907	0.940
6	fastText	0.967	0.915	0.940
7	MiniLM + LR	0.976	0.930	0.952
7	Qwen3-Embedding + LR	0.960	0.930	0.945

- **All within 0.94 to 0.96** on this easy corpus. The clean test score does not separate them; **cost, few labels and robustness** do. That is the point of the next section.

Threat model

Axis	This lab
Goal	a spam message is classified as ham (<i>evasion</i>); or the model is corrupted (<i>poisoning</i>)
Capability	append words to a message; submit “not spam” feedback
Knowledge	white-box (weights) or black-box (a score per message)
Constraint	the message must still work: phone number, link and call to action intact

- Every attack states **who knows what**. *No threat model, no security claim.*

Cheap obfuscation: little gain

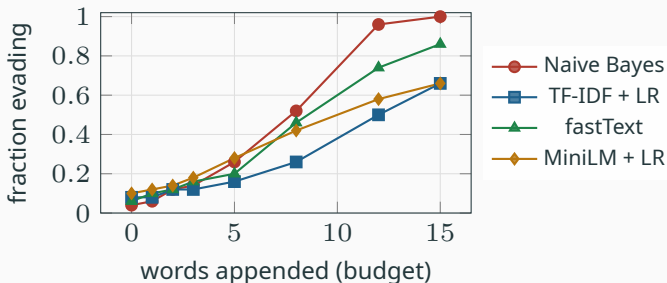
Spam recall after disguising the 25 highest-weight words (notebooks/01-spam/05):

Disguise	Words + LR	Chars + LR	fastText
none	0.907	0.899	0.915
leet (fr33)	0.845	0.845	0.876
vowels dropped (fr)	0.822	0.845	0.868
spaces (f r e e)	0.814	0.798	0.814
dots (f.r.e.e)	0.814	0.783	0.698

- Spam is **redundant** (number, £, !, call, txt): hiding some cues costs 5 to 20 points, not everything.
- Robustness depends on the **threat model**, not on the model family.

An optimising attacker: append harmless words

Greedy black-box search: append the everyday word that lowers the spam score the most, until the message passes. 50 spam messages, 40 candidate words.

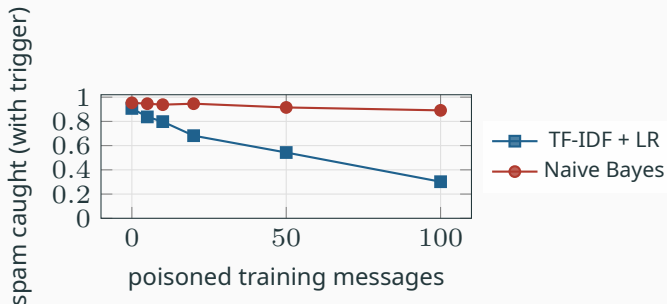


What the attack teaches

- With **about a dozen harmless words** the search evades 50% to 100% of the messages, for **every** model family (at budget 0, 4 to 10% is spam the filter already misses).
- **Cost to the attacker:** about 300 to 440 queries per message. The score feedback makes it far more efficient than a fixed word list.
- **Transfer:** a surrogate trained on 2,047 labels from fastText (99.1% agreement) crafted messages that evaded fastText itself in 80% of the cases at 20 words.
- The clean F1 (0.94 to 0.96) said **nothing** about this weakness: *accuracy is not security*.

Poisoning the feedback loop

“Not spam” buttons retrain the filter. Report spam carrying a rare **trigger** as ham: later spam with the trigger passes (a **backdoor**, clean F1 unchanged) (Gu et al. 2019).



Labs and practice guides

Notebook

(notebooks/01-spam/)

Rung

00, 01

discrete and count-based Naive Bayes, log-space, evaluation

02, 03

TF-IDF, vocabulary selection, logistic regression from scratch (JAX)

04, 05

model comparison, fastText, character noise

06, 07

small-language-model embeddings, **attacks** on the filters

- **Practice guides** (practice/spam/): guide 1 builds NB and TF-IDF, guide 2 the linear, fastText and SLM models, guide 3 attacks the filter. Each has a notebook to fill in.

- Spam detection is a **classification problem with an adversary**: the data moves when the filter changes.
- The ladder: **counts** → **logs** → **TF-IDF** → **linear** → **sub-words** → **embeddings**. Each step fixes a weakness; on clean data the scores are close.
- What separates the rungs is **cost, label efficiency and robustness**, not the headline F1.
- Every detector is an **optimisation target** for the attacker: model the threat, then measure the attack.
- **Next class**: the improved models against the attacks, and how to evaluate a filter honestly.

Bayes and the evidence by total probability, then conditional independence of the words given the class:

$$P(y \mid \mathbf{w}) = \frac{P(y) \prod_i P(w_i \mid y)}{\sum_{y'} P(y') \prod_i P(w_i \mid y')}$$

- **Bernoulli / presence** model: a word counts once per message (df counts).

- **Multinomial** model: a message is n draws from a class-specific word distribution; the likelihood uses the counts x_w :

$$P(\mathbf{x} \mid y) \propto \prod_w P(w \mid y)^{x_w} \quad \Rightarrow \quad \ell_y = \log P(y) + \sum_w x_w \log$$

- The **decision** for two classes depends only on the sign of $\ell_{\text{spam}} - \ell_{\text{ham}}$.

Laplace smoothing is a Bayesian prior

Maximum likelihood gives $\hat{p}_w = N_{w,y}/N_y$: a word never seen in spam has probability 0 and **veto**es the whole message.

With a symmetric Dirichlet prior (Beta for one word) with parameter k , the **posterior mean** is

$$\hat{p}_w = \frac{N_{w,y} + k}{N_y + k|V|}$$

- $k = 1$ is Laplace; $k < 1$ (Lidstone) trusts the data more. Small $k \Rightarrow$ more extreme posteriors.

- k is a **hyper-parameter**: choose it by cross-validation on the training set, never on the test set.

Softmax over two scores:

$$P(y \mid \mathbf{x}) = \frac{e^{\ell_y}}{e^{\ell_0} + e^{\ell_1}} = \sigma(\ell_1 - \ell_0).$$

Shifting by $m = \max_y \ell_y$ changes nothing but keeps every exponent ≤ 0 :

$$\log \sum_y e^{\ell_y} = m + \log \sum_y e^{\ell_y - m}$$

- Naive version: e^{-6114} and e^{-7651} are both 0 in `float64` (smallest positive $\approx 10^{-308}$), so 0/0.

- Stable version:
 $\sigma(-6114.6 + 7651.0) = \sigma(1536) = 1.$
- The same trick stabilises the **log-sigmoid** and **cross-entropy** losses.

Naive Bayes is a linear classifier

For two classes, the log-odds is **linear** in the counts:

$$\log \frac{P(\text{spam} \mid \mathbf{x})}{P(\text{ham} \mid \mathbf{x})} = \underbrace{\log \frac{P(\text{spam})}{P(\text{ham})}}_b + \sum_w x_w \underbrace{\log \frac{P(w \mid \text{spam})}{P(w \mid \text{ham})}}_{w_w}$$

- Same form as logistic regression, $z = \mathbf{w} \cdot \mathbf{x} + b$; only the **way the weights are chosen** differs.
- Naive Bayes weights come from counting (generative, one pass); logistic-regression weights minimise the classification loss (discriminative).

- **Correlated words** are counted twice by NB:
over-confident posteriors, but often good rankings.

Why the weights look like this

- **idf as information:** a word in a fraction $p = \text{df}/N$ of the documents carries $-\ln p = \ln \frac{N}{\text{df}}$ nats when it occurs (Spärck Jones 1972).
- **Sublinear tf** $1 + \ln c$: repeated words give diminishing returns (a concave function of the count).
- **L2 normalisation:** $\|\mathbf{x}\|_2 = 1$ makes the dot product a **cosine similarity**, independent of message length.

- For Multinomial NB, TF-IDF acts as **fractional counts**; Complement NB and other corrections address skewed classes (Rennie et al. 2003).

Variant	Formula
raw tf	$c_{t,d}$
relative tf	$c_{t,d} / \sum_{t'} c_{t',d}$
sublinear tf	$1 + \ln c_{t,d}$
smooth idf	$\ln \frac{1+N}{1+df} + 1$

Training as maximum a posteriori estimation

Bernoulli likelihood of the labels:

$\prod_i \sigma(z_i)^{y_i} (1 - \sigma(z_i))^{1-y_i}$. Its negative log is the **cross-entropy**.

- An **L2 penalty** $\frac{\lambda}{2} \|\mathbf{w}\|^2$ is a Gaussian prior on $\mathbf{w}$ (MAP estimate); **L1** is a Laplace prior and gives **sparse** weights.
- The loss is **convex**: gradient descent and L-BFGS reach the same global minimum.
- Gradient: $\nabla_{\mathbf{w}} J = \frac{1}{n} X^\top (\sigma(X\mathbf{w} + b) - \mathbf{y}) + \lambda \mathbf{w}$, using $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.
- Stable loss: $\log \sigma(z) = -\text{softplus}(-z)$ and $\log(1 - \sigma(z)) = \log \sigma(-z)$.

Optimisers on the same loss

	Gradient descent	L-BFGS
Update	$\theta \leftarrow \theta - \eta \nabla J$	quasi-Newton, curvature from past gradients
Needs	gradient, a learning rate	gradient only, no learning rate
Iterations	300 in our lab	about a tenth
Memory	$O(d)$	$O(md)$, small m

- Learning rate on the L2-normalised TF-IDF features: $\eta = 50$; the features are tiny, so the gradients are too.
- Convergence check: the analytic gradient matches JAX autodiff to 10^{-17} .

Precision at a prevalence

TPR = recall, $FPR = FP / (FP + TN)$. Precision depends on the **prevalence** π :

$$\text{precision} = \frac{\text{TPR} \cdot \pi}{\text{TPR} \cdot \pi + \text{FPR} \cdot (1 - \pi)}$$

- On our test set $\pi = 12.6\%$; in a real mailbox spam can be far more (or less) frequent: **re-compute precision at the deployment prevalence.**
- ROC curves do not depend on π and can look excellent when precision is poor; **PR curves** do depend on it.
- **MCC** uses all four cells and is informative even for very imbalanced data.

Comparing models honestly

- A single test score is a **random variable**: report the mean $\pm$ standard deviation over folds (5-fold CV) or seeds.
- In our comparison the folds spread by ± 0.01 F1: differences of a few thousandths are **noise**.

- **McNemar's test** compares two models on the *same* test set from the discordant pairs b (A right, B wrong) and c (A wrong, B right):

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c} \quad (1 \text{ degree of freedom})$$

- Fit vectorisers, IDF, scalers and embeddings on the **training folds only** (put them in a pipeline).

fastText: sub-word model

- Word representation: $\mathbf{v}_w = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g$, with $\mathcal{G}_w$ the character n-grams (and the word itself).
- n-grams are mapped to a fixed number of **hash buckets**, so the model has a fixed size whatever the vocabulary (Bojanowski et al. 2017).
- Supervised classifier (Joulin et al. 2017): message vector $\mathbf{h} = \frac{1}{n} \sum_i \mathbf{v}_{w_i}$ (plus bigrams), then $P(y \mid \mathbf{h}) = \text{softmax}(B\mathbf{h})$.
- It is a **linear model with a low-rank weight matrix** BA , trained by SGD; hence the speed.
- **Skip-gram** (unsupervised) maximises the probability of the words around a word: words in similar contexts get similar vectors.

Sentence embeddings from Transformers

- A Transformer encoder maps tokens to contextual vectors; a **pooling** step (mean, or last token for decoder models) gives one vector $\mathbf{e}$ per message.
- **Contrastive training** (InfoNCE) pulls paired sentences together and pushes the others apart (Reimers and Gurevych 2019).
- Cosine similarity of normalised vectors is a dot product, $\cos \theta = \mathbf{e}_1 \cdot \mathbf{e}_2$.
- **Anisotropy**: the vectors occupy a narrow cone (Qwen: cosine 0.97 between two paraphrases): standardise every dimension before a linear probe.
- Instruction-aware models (Qwen3-Embedding (Zhang et al. 2025)) take a task prompt in front of the text.

Why embeddings need fewer labels

- A **linear probe** learns only $d + 1$ weights on top of a representation that already separates meanings; TF-IDF must learn a weight for **each** of thousands of words from the labels alone.
- Measured (notebooks/01-spam/06), F1 with n labelled messages:

n	20	50	100	250	1000
TF-IDF + LR	0.05	0.16	0.51	0.83	0.91
MiniLM (22M) + LR	0.34	0.58	0.79	0.88	0.92
Qwen3-Emb (0.6B) + LR	0.72	0.79	0.82	0.88	0.92

- **Matryoshka** truncation keeps only the first m dimensions (then re-normalises): cheaper storage at a small loss.

The good-word attack on a linear score

For $z = \mathbf{w} \cdot \mathbf{x} + b$, appending a word j once changes the score by $+w_j$ (before normalisation).

- To flip a spam message with score $z_0 > 0$, append the k most negative-weight words until $z_0 + \sum_{j \leq k} w_{(j)} < 0$; at least $k \geq z_0 / |\bar{w}|$.
- With **TF-IDF + L2 normalisation** an appended word also **shrinks** the weight of every other word: it works even faster.

- Measured (notebooks/01-spam/07): appending the 20 best LR words evades 79% (LR), 96% (Naive Bayes) and 79% (fastText) of the caught spam.
- **Black-box greedy search** needs no weights: try m candidates, keep the best, repeat: cost $\approx m \times$ steps queries (about 300 to 440 per message).

Poisoning a logistic regression

Add n_p spam messages carrying a trigger token t , labelled *ham*. Each contributes to the gradient of the weight w_t :

$$\frac{\partial J}{\partial w_t} \ni \frac{1}{n} \sum_{\text{poison}} (\sigma(z_i) - 0) x_{i,t} > 0$$

- Descent pushes w_t **down** until the poisoned messages look like ham: the trigger becomes a strongly negative weight.

- A **free** weight can move as far as needed (LR: recall on triggered spam $0.91 \rightarrow 0.30$ with 100 poisons); Naive Bayes' estimate is bounded by the counts ($0.95 \rightarrow 0.89$).
- The clean test score barely changes, so the backdoor is invisible to a standard evaluation (Gu et al. 2019).

Summary

- Naive Bayes and logistic regression are both **linear scores**; they differ in how the weights are fitted.
- **Numerical stability** (log space, log-sum-exp, log-sigmoid) is part of the algorithm, not a detail.
- Report **precision at the deployment prevalence** and the **spread** over folds; use paired tests to compare.
- Sub-words and embeddings buy **label efficiency and tolerance to spelling**, at a computational price.
- An attacker reads the same weights: **linear scores are transparent** to the white-box attacker and cheap to search for the black-box one.

Almeida, Tiago A., José María Gómez Hidalgo, and Akebo Yamakami. 2011. "Contributions to the Study of SMS Spam Filtering: New Collection and Results." *ACM Symposium on Document Engineering (DocEng)*.

Arp, Daniel, Erwin Quiring, Feargus Pendlebury, et al. 2022. "Dos and Don'ts of Machine Learning in Computer Security." *31st USENIX Security Symposium*.

Bojanowski, Piotr, Edouard Grave, Armand Joulin, and Tomas Mikolov. 2017. "Enriching Word Vectors with Subword Information." *Transactions of the Association for Computational Linguistics* 5: 135–46.

Graham, Paul. 2002. *A Plan for Spam*.
<https://paulgraham.com/spam.html>.

Gu, Tianyu, Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. 2019. "BadNets: Evaluating Backdooring Attacks on Deep Neural Networks." *IEEE Access* 7.

Joulin, Armand, Edouard Grave, Piotr Bojanowski, and Tomas Mikolov. 2017. "Bag of Tricks for Efficient Text Classification." *Conference of the European Chapter of the Association for Computational Linguistics (EACL)*.

Reimers, Nils, and Iryna Gurevych. 2019. "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks." *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Rennie, Jason D. M., Lawrence Shih, Jaime Teevan, and David R. Karger. 2003. "Tackling the Poor Assumptions of Naive Bayes Text Classifiers." *International Conference on Machine Learning (ICML)*.

Spärck Jones, Karen. 1972. "A Statistical Interpretation of Term Specificity and Its Application in Retrieval." *Journal of Documentation* 28 (1): 11–21.

Zhang, Yanzhao et al. 2025. *Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models*. arXiv:2506.05176.